Threadplane

# Designing Agent Experiences

Help people understand, control and recover agent work.

Threadplane · Open source · Angular + React

Engineering guide · 2026 · Research reviewed November 7

threadplane.ai

# Inside this guide

Practical boundaries, current APIs, and decisions to make before you ship.


01 Design around the user's job

02 Show progress that corresponds to observable work

03 Make evidence inspectable and conclusions revisable

04 Separate proposal, decision, and outcome

05 Make return and recovery first-class journeys

06 Preserve control across layouts and input modes

07 Measure completion, recovery, and calibrated trust

08 Dated implementation and reference notes


**For senior engineers and technical leaders**

Framework-neutral architecture, with Angular and React examples. Published Angular adapters and native source-build bindings are distinct profiles; each example identifies its scope. Native packages remain private and are not npm installation instructions.

Research and repository API review: November 7, 2026. Model catalogs and upstream specifications are dated observations, not a promise of Threadplane support. Threadplane is free and open source; you own your backend, data and deployment.

# Design around the user's job

An agent experience succeeds when people can understand what is happening, judge the evidence, exercise meaningful control, and recover when work is interrupted. A transcript is one useful representation of that work. It is rarely the whole product. The interface should make the task easier to complete, rather than require users to manage a conversation as a substitute for an application.

The running example is a hypothetical invoice investigation. A finance analyst asks whether invoice INV-1042 duplicates an earlier charge. The assistant gathers evidence and proposes an illustrative USD 480.00 credit. The analyst reviews the proposal, approves or declines it, and receives the finance service's confirmed outcome. All identifiers and amounts are invented for the example; no measured customer result is implied.

## Choose a surface that fits the decision

| Surface | Useful when | Design responsibility |
| --- | --- | --- |
| Conversational view | The question is open-ended and refinement matters | Keep the current task and outcome visible within the transcript |
| Embedded assistant | Help is secondary to an existing application task | Preserve the surrounding context and avoid obscuring controls |
| Task workspace | Users must compare evidence, decide, and track results | Coordinate conversation, artifacts, decisions, and history |
| Conventional form or table | Inputs and operations are already known | Keep the direct path efficient and predictable |

For invoice investigation, a task workspace is a strong candidate. The transcript explains the investigation; an evidence region compares records; a decision region presents the pending proposal. The analyst should not have to scroll through a long conversation to discover whether a credit needs approval.

For a simple invoice-number lookup, a search field and result table may be better. The presence of a model elsewhere in the product does not require natural language for every action. Offer a direct route when the user already knows the identifier or desired operation.

## Make the starting contract visible

The initial screen should explain what the assistant can investigate, which records it can use, and which actions require a decision. A useful prompt suggestion is specific enough to reveal the task: “Compare this invoice with earlier charges for the same service period.” A vague invitation to ask anything makes the user's burden larger.

Keep the input purpose visible after typing begins. Label attachments and distinguish selecting a file from sending it into an investigation. If the user supplies incomplete information, identify the missing field or evidence rather than relying on a generic failure toast. The accessibility foundation is ordinary input design: labels and instructions should make the expected action understandable. Primary reference: W3C labels or instructions.

# Show progress that corresponds to observable work

Streaming text can reassure users that a request is active, but it is only one signal. The investigator may be searching records before it has anything useful to say. It may finish its explanation while waiting for approval. It may stop producing text while a tool continues. A useful progress model reflects those differences.

## Separate composing, accepting, and executing

The draft belongs to the user until the application accepts the request. During acceptance, preserve enough state to explain a rejection without losing work. Once execution starts, show the current investigation and provide only the controls supported by the runtime and application.

A practical sequence for the invoice task is: request accepted, searching invoices, comparing service periods, proposal ready, waiting for decision, submitting approved action, and outcome confirmed. These labels describe observable stages. They should not imply a measured percentage complete unless the system actually has a meaningful denominator.

Do not fabricate a private reasoning transcript to fill silence. A concise explanation such as “Checking whether a credit already exists” is useful when it corresponds to an actual tool operation or declared workflow stage. A model-generated claim that it has carefully verified everything is not an execution record.

## Make waiting informative

If a stage takes longer than expected, identify what is still pending. For example, “Waiting for the invoice service” is more actionable than an endless animated ellipsis. When a known timeout occurs, explain the affected part of the task and the supported next step. Keep timing thresholds based on measured service behavior, not arbitrary animation intervals.

Allow the analyst to inspect already available evidence while additional retrieval proceeds. Do not make every region of the workspace unusable because one operation is pending. At the same time, keep the approval action unavailable until its required evidence and proposal state are ready.

## Describe cancellation accurately

Stop can mean several things: stop displaying a stream, request remote cancellation, or abandon a local draft. Choose a label and explanation that match the actual command. If cancellation is pending, show that state. If the credit already committed, closing the stream cannot undo it.

Angular and React can both derive progress from an application-owned snapshot. Neither framework makes a run complete when its component disappears. Threadplane's Angular streaming guide and React streaming guide provide implementation-specific context.

Announce important status changes without forcing focus away from the user's work. Streaming every token through an assertive live region can make the application harder to use. Choose concise, meaningful updates and test them with assistive technology. Primary reference: W3C status messages.

# Make evidence inspectable and conclusions revisable

An assistant's explanation, a retrieved record, a tool observation, and an execution receipt are different kinds of information. Presenting them with identical visual weight can make a plausible narrative seem as authoritative as the finance system. Users need enough provenance to decide what deserves trust.

## Put the evidence beside the claim

In the invoice comparison, show the two invoice identifiers, amounts, currencies, service periods, and purchase-order references near the conclusion. If the service periods differ, make that contradiction visible. If a purchase order is missing, say so before presenting a proposed credit. Do not bury the main uncertainty in a collapsed technical trace.

A useful citation identifies the underlying record and lets an authorized user inspect it. A generated label that looks like a citation but does not resolve to evidence is misleading. Retain the source version or retrieval time when freshness matters. If access has changed since the investigation, the record link should reflect the current authorization result without leaking protected details.

The explanation can say why the evidence suggests a duplicate, but should distinguish inference from a confirmed accounting fact. For example, identical totals and descriptions may support suspicion while an existing credit could invalidate the proposed action. The analyst needs the unresolved question, not just the most persuasive sentence.

## Give tool observations a useful level of detail

A collapsed observation can state that two invoice searches completed and one purchase-order lookup failed. An expanded view can reveal the relevant parameters and result summary. Keep raw partial arguments separate from accepted requests, and do not expose sensitive credentials or unrelated record contents to make the view appear transparent.

The user-facing evidence view and the operator trace serve different audiences. An operator may need run identifiers, timing, and adapter events. The analyst needs the invoice comparison and the next supported action. Observability systems can help connect the records without making the full trace part of the everyday interface. Primary reference: LangSmith observability.

### Treat artifacts as durable outcomes

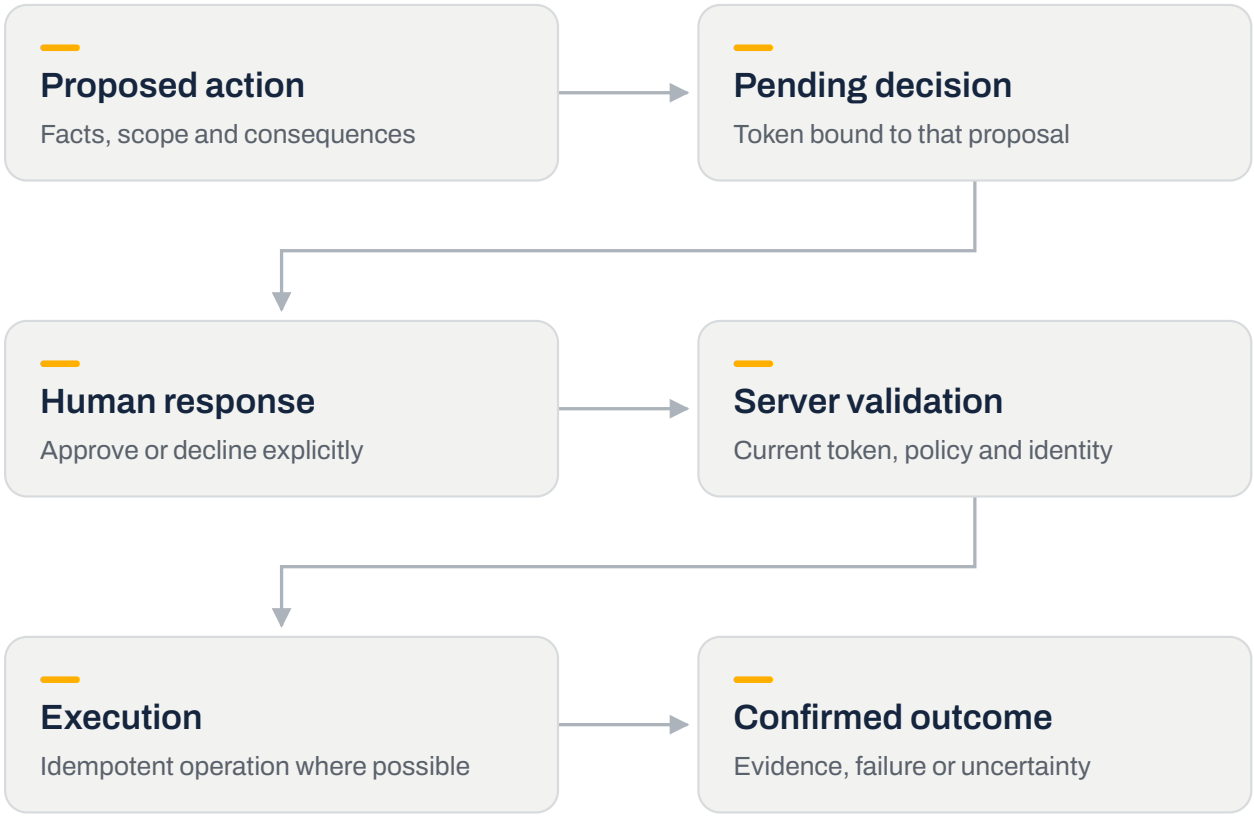
A comparison report or credit receipt should remain findable after the surrounding conversation grows. Give artifacts stable titles and an explicit relationship to their source task. If a report is regenerated after new evidence arrives, preserve enough version context that the user can tell which proposal it supported.

An artifact can also be wrong. Provide a route to correct or challenge the underlying conclusion without pretending that editing its displayed text changes the system of record. The application should distinguish a revised explanation, a revised proposal, and a new business operation. This distinction keeps the interface useful when the first answer needs repair.

## Separate proposal, decision, and outcome

Approval is most understandable when the interface treats it as three stages. A proposal explains the exact action and its basis. A decision records the user's intent about that proposal. An outcome reports what the authoritative service actually did. Collapsing those stages into one button creates ambiguity precisely where users need confidence.

## Approval is not completion




Approval, decision acceptance and operation completion are different states. A decline ends this action path. Stale or unauthorized responses are rejected before execution.

For the invoice example, proposal P-7 identifies INV-1042, USD 480.00, the affected account, and the supporting comparison. The analyst chooses approve or decline. If approved, operation OP-23 is submitted to the finance service. Its receipt confirms the credit. The proposal card remains a record of the decision even if execution later fails or requires reconciliation.

## Make the decision specific

Use action language that reveals the consequence. A credit proposal should expose the amount, currency, and target before the user decides. Give decline a clear meaning: it rejects this proposal, rather than silently canceling every investigation in the thread. If editing is supported, changing a material field should produce a new proposal revision and a new decision opportunity.

A decision token binds the interaction to a particular pending proposal. It is not authorization by itself. The backend must check the user's authority, proposal version, expiry, and prior decisions. Two people may view the same proposal; the second response must resolve against the current authoritative state.

## Keep view components focused on intent

These paired examples are application-owned decision interfaces. The Angular component emits a token through approve or decline outputs. The React component calls a supplied response callback with the token and choice. Their `canRespond` value controls the displayed affordance. Neither component authorizes an action, resumes a runtime, or proves completion.

```
import { Component, input, output } from '@angular/core';

// Application contract, not a Threadplane session method.
interface Decision {
  readonly token: string;
  readonly summary: string;
  readonly canRespond: boolean;
}

@Component({
  selector: 'app-approval',
  standalone: true,
  template: `
    <p>{{ decision().summary }}</p>
    <button [disabled]="!decision().canRespond"
      (click)="approve.emit(decision().token)">Approve</button>
    <button [disabled]="!decision().canRespond"
      (click)="decline.emit(decision().token)">Decline</button>
  `,
})
export class Approval {
  readonly decision = input.required<Decision>();
  readonly approve = output<string>();
  readonly decline = output<string>();
}
```

```
// Application contract, not a Threadplane session method.
interface Decision {
  readonly token: string;
  readonly summary: string;
  readonly canRespond: boolean;
}

export function Approval({ decision, respond }: {
  decision: Decision;
  respond: (token: string, choice: 'approve' | 'decline') => void;
}) {
  return <section aria-label="Proposed action">
    <p>{decision.summary}</p>
    <button disabled={!decision.canRespond}
      onClick={() => respond(decision.token, 'approve')}>Approve</button>
    <button disabled={!decision.canRespond}
      onClick={() => respond(decision.token, 'decline')}>Decline</button>
  </section>;
}
```

The surrounding application maps that intent to the actual backend capability, tracks submission, and refreshes the decision state. It must validate the incoming proposal shape before rendering meaningful fields. A disabled button is useful interaction feedback, but the server remains responsible for rejecting duplicate or stale decisions.

### Explain the intermediate states

After approval, show that the decision was accepted or is being checked, as appropriate. Do not immediately replace the card with “Credit issued” unless the service has supplied that outcome. If a connection drops, retain the proposal and explain that the result needs checking. If the proposal expired, show that it can no longer be decided and offer a supported route to a refreshed investigation.

A confirmation surface is one part of preventing consequential mistakes; it is not a blanket accessibility or transaction-safety guarantee. The applicable accessibility guidance discusses review, correction, and reversal mechanisms for important submissions. Primary reference: W3C error prevention for legal, financial, and data actions. Runtime-specific pause and resume behavior is documented in LangGraph interrupts.

## Make return and recovery first-class journeys

Users leave tasks unfinished for ordinary reasons: another meeting, a closed laptop, a different device, or a lost connection. The return journey should explain what changed while they were away and whether anything requires a decision. Restoring a transcript alone may leave the most important state invisible.

### Reopen the task, not just its text

When the analyst returns, load the authorized thread, current proposal, and operation status. If P-7 still waits for a decision, make that visible. If another analyst declined it, show the established result. If OP-23 completed, display the receipt even if the original stream never delivered its final message.

Keep navigation state distinct from persistence. A browser can remember a thread address, but the server determines whether that thread exists and is accessible. Explain deleted, expired, or unavailable conversations without treating them as fresh empty ones. Avoid revealing another user's title or evidence while reporting an authorization failure.

A historical view should be visibly historical. A checkpoint can help explain how the investigator reached a proposal, but it is not automatically an actionable current state. Branching and replay should only appear when the runtime and application support them, with a clear explanation of whether the operation creates a new investigation or changes the active one.

### Recover unknown outcomes deliberately

Suppose the finance service commits OP-23 and the connection fails before the browser receives the receipt. The user sees “Checking the credit outcome.” A read-only status request then resolves the operation and returns the receipt. This is a successful recovery, even though the original streaming journey was interrupted.

If status cannot be obtained yet, preserve the unknown state. Offer a later check or a supported human escalation path. Do not present a generic Retry that might issue another credit. When replay is appropriate for a safe read, label it according to the actual work being repeated.

### Make errors useful at the point of repair

An invalid invoice identifier belongs near the input. A retrieval permission problem belongs near the missing evidence. An uncertain credit outcome belongs near the operation. A single global error banner rarely gives enough context for all three. Describe what is known, what remains uncertain, and the action the user can take.

Preserve useful drafts and evidence when a recoverable error occurs. Avoid resetting the whole workspace merely to clear an error flag. W3C guidance on identifying input errors is a foundation for clear repair paths; the recovery policy for remote operations must be designed separately. Primary reference: W3C error identification. Persistence implementation references: Angular and React.

## Preserve control across layouts and input modes

A task workspace must remain understandable on a narrow screen, with enlarged text, through keyboard navigation, and with assistive technology. These are part of the interaction design, not a finishing pass after streaming works. The invoice comparison, pending decision, and recovery control must remain associated even when the layout changes.

### Design the order of attention

On a wide screen, the transcript, evidence, and decision can occupy adjacent regions. On a narrow screen, the same information may need a sequence of sections or explicit navigation. Choose a reading and focus order that preserves the relationship between the proposed action and its evidence. Do not let a visually prominent mobile button appear before the context needed to understand it.

Streaming should respect the reader's position. If the analyst scrolls upward to inspect an earlier invoice, new text should not repeatedly pull them to the bottom. Provide a visible way to return to the latest content. Preserve element identity so ordinary updates do not unexpectedly reset an expanded comparison or move focus.

Keyboard focus should follow a meaningful sequence through interactive content. If a user explicitly opens a modal decision dialog, manage focus according to that pattern and restore it sensibly on close. A non-modal evidence panel has different behavior and should not borrow modal semantics merely for convenience. Primary references: W3C focus order and ARIA modal dialog pattern.

### Keep design-system behavior consistent

Map chat and generated surfaces to the application's established colors, typography, spacing, and focus treatment. Evaluate real content: a long invoice number, a dense comparison, a disabled approval, an error, and a citation. A pleasant empty-state theme does not establish readability under those conditions.

Angular and React have different component and theming surfaces. The Angular theming guide and React theming guide are implementation references, not a promise of identical component inventories. Preserve product semantics while choosing the supported composition in each framework.

### Treat multimodal entry as another review boundary

A voice request or uploaded invoice can make entry easier, but it introduces interpretation and processing states. Show what was captured, let users correct material information, and distinguish upload completion from successful extraction. A spoken “four eighty” should not silently become an approved monetary amount.

The final proposal should present a clear, reviewable value regardless of the original input mode. Maintain a usable text path and explain unsupported file or modality cases. These are general product recommendations; they do not claim that every Threadplane adapter or chat component supplies all multimodal capabilities.

## Measure completion, recovery, and calibrated trust

A useful experience measurement asks whether users reached the right outcome with appropriate understanding. Message counts, time spent chatting, or the number of generated cards can rise while the product gets worse. Define measures around the task and inspect their failure cases.

## Measure the journey at meaningful boundaries

Time to useful output can measure when the first relevant evidence becomes available. Decision time can measure how long users need to evaluate a complete proposal. Task completion can measure confirmed credits or justified decisions not to act. Recovery success can measure whether interrupted work reaches a known outcome without duplication or unnecessary restart.

These are proposed measures, not reported results. Establish targets from user research and operational requirements. Segment them by task difficulty, missing evidence, and service failures. An average that combines obvious duplicates with ambiguous disputes can conceal where the experience needs improvement.

Trust calibration is visible in behavior: users should accept well-supported conclusions and challenge unsupported ones. Test both. A design that makes every proposal feel authoritative may increase approval speed while reducing decision quality. Presenting uncertainty well can sometimes slow an individual decision and improve the outcome.

## Connect tests without conflating them

| Evidence | What it should establish |
| --- | --- |
| Component and accessibility checks | Controls, focus, labels, and status behavior for known states |
| Scripted interaction tests | Coherent transitions through progress, approval, failure, and return |
| Adapter and backend integration | Real command mapping, authorization, persistence, and reconciliation |
| Model evaluations | Relevant investigation, correct evidence use, and appropriate stopping |
| User research | Understanding, effective control, and successful task completion |

For the invoice journey, require a visible pending decision after reload, rejection of stale approval, a useful explanation of missing evidence, and a receipt after successful reconciliation. Verify that a lost response cannot be mistaken for permission to repeat the credit. Test Angular and React views against the same domain expectations while using their own supported APIs.

Maintain a set of representative recorded or synthetic journeys for regression review. Label synthetic evidence clearly and avoid treating a simulator as proof of production persistence. Calibrate any model-assisted experience grading against human judgments, particularly when the criterion concerns understanding rather than exact fields. Primary reference: Anthropic, Demystifying evals for AI agents.

# Dated implementation and reference notes

Checked on 7 November 2026. This paper's invoice workflow and metrics are illustrative design proposals. The accessibility sources support specific design practices; this paper does not certify a product or PDF as conforming to an accessibility standard.

## Framework scope

Threadplane's published-line Angular packages are version 0.3.7. Native core, Angular, and React packages are private at 0.0.0 and use a source-build profile. Native Angular and React observers borrow an application owner; component teardown releases only their subscription. The current native React observer has no server snapshot and makes no SSR claim.

The paired approval examples define an application decision contract. They do not add a universal resume method to the minimal core session. Rich React chat exports and native Angular chat exports differ; a shared product journey does not imply universal widget parity. Start with the Angular chat quickstart and React introduction, then verify each required capability.

The native repository baselines are Angular 21.1.6 and React 19.2.4. Checked upstream releases were Angular 22 and React 19.3. Release sources: Angular and React. Connecting Agents to Applications covers the ownership and transport contracts beneath this experience; Generative Interfaces covers the controlled rendering of adaptive evidence views.